

SKILLREVISE: Improving LLM-Authored Agent Skills via Trace-Conditioned Skill Revision

Yuxuan Liu¹ Zhaochen Su¹ Lingyun Xie² Yuhao Zhang³ Qing Zong¹
 Jiahe Guo² Zhongwei Xie¹ Yiyan Ji⁴ Yauwai Yim¹ Hongyu Luo¹
 Xiyu Ren¹ Ruan Chenyu⁵ Haoran Li^{1*} Yangqiu Song¹

¹The Hong Kong University of Science and Technology ²Harbin Institute of Technology

³Harbin Institute of Technology, Shenzhen ⁴Nanjing University ⁵The University of Hong Kong

{yliurk,zsubf,hlibt*}@connect.ust.hk, yqsong@cse.ust.hk

Code: <https://github.com/HKUST-KnowComp/skillrevise>

Abstract

Agent skills are procedural artifacts that enable LLM agents to execute workflows, verify constraints, and recover from failures. Existing self-evolving methods refine skills using accumulated trajectories. However, they struggle in cold-start settings, where only an initial, imperfect skill is available. Consequently, skill construction defaults to expert authoring or one-shot LLM generation. Expert-authored skills are costly and may not align with how LLM agents actually execute tasks, while one-shot generated skills can be syntactically well formed yet behaviorally weak. To bridge this gap, we propose SKILLREVISE, an execution-grounded framework designed to iteratively refine these initial skills. SKILLREVISE diagnoses skill defects from execution evidence, retrieves relevant repair principles from a general memory, and applies execution-anchored edits. By re-executing candidates, it retains the first verifier-passing skill within the revision budget and falls back to empirical utility only when no candidate succeeds. Evaluated across three benchmarks and five LLMs, SKILLREVISE substantially outperforms one-shot baselines—improving the base agent’s success rate on SkillsBench from 36.05% to 61.63%. Furthermore, the revised skills transfer across both executors and task environments, suggesting that SKILLREVISE captures reusable procedural knowledge beyond any single executor.

1 Introduction

LLM agents are increasingly expected to operate in complex, verifier-driven environments rather than merely produce natural-language answers (Li et al., 2026). In such settings, success requires agents to inspect workspaces, invoke tools, generate artifacts, satisfy task-specific constraints, and recover from failures (Zhou et al., 2026). Existing tools and prompts provide useful capabilities and high-level guidance, but they do not fully specify when

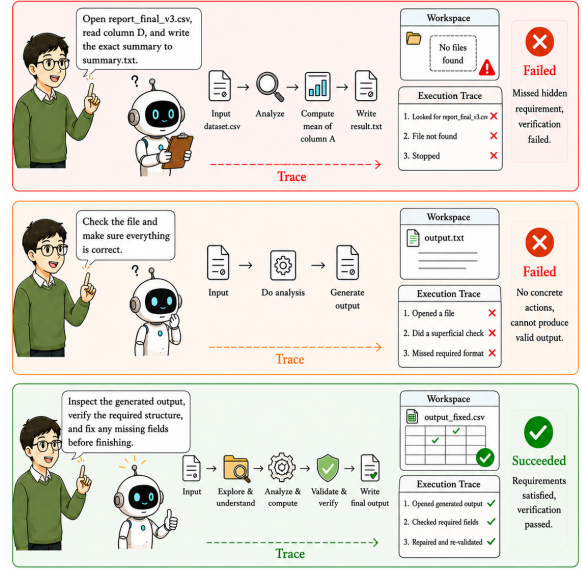


Figure 1: Skill design must avoid both instance-specific shortcuts and vague advice; trace-conditioned principles provide actionable, verifier-aligned guidance.

to act, how to sequence operations, how to validate outputs, or how to handle errors. *Agent skills* address this gap by externalizing reusable procedural knowledge: unlike atomic tools or one-off prompts, skills organize multi-step workflows, execution constraints, verification checkpoints, and recovery strategies (Anthropic, 2025). Figure 1 illustrates the central design tension: overly instance-specific skills can encode brittle shortcuts, while overly generic skills fail to guide concrete execution; useful skills should instead abstract execution traces into actionable, verifier-aligned principles.

However, skills do not improve agent performance as reliably as atomic tools (Li et al., 2026; Han et al., 2026). Tools typically expose well-defined operations with clear input-output behavior. A skill, by contrast, guides how an agent should organize its behavior; its value depends not only on the written instruction itself, but also on how

the skill is selected, triggered, executed, and maintained in the target environment (Liu et al., 2026b). Recent studies on agent skills show that low-quality or poorly matched skills often yield little improvement (Liu et al., 2026b) and may even degrade agent performance (Li et al., 2026; Han et al., 2026; Gao et al., 2026), shifting the focus from skill adoption to skill acquisition.

Existing approaches to skill acquisition can be broadly grouped into three categories: retrieval, self-evolution, and direct authoring (Zhou et al., 2026). Retrieval-based methods reuse skills from existing repositories or memory stores. However, the selected skills may not fully match the target task and result in disrupted execution and poor adaptivity (Liu et al., 2026b; Han et al., 2026). Self-evolution methods refine skills from an agent’s own trajectories, failures, feedback, and past behavior (Wang et al., 2026; Ma et al., 2026). However, self-evolution requires sufficient experience and can suffer from overfitting or cold-start problems in new settings. Direct authoring or generation creates skills from expert knowledge or task-conditioned LLM outputs, which is useful when no appropriate skill is available (Anthropic, 2025; Li et al., 2026). Nevertheless, expert authoring is costly, and one-shot generated skills may lack behavioral reliability without execution-based validation (Li et al., 2026).

To bridge this gap, we propose SKILLRE-
VISE, a revision framework built around a task-specific/general decomposition. Inspired by agent-learning methods that separate task-specific experience from general reusable knowledge (Xia et al., 2026; Ren et al., 2026), we treat revision as coupling task-specific execution evidence with general repair knowledge. Diagnosis identifies what failed and what should be preserved in the current episode; retrieved repair principles capture recurring design defects. Starting from an initial LLM-authored skill, SKILLRE-
VISE executes, diagnoses, revises, re-executes, and selects the first verifier-passing version under a bounded budget, using empirical utility only as a fallback when all evaluated candidates fail. Thus, it keeps the feedback advantage of self-evolution without requiring a large skill corpus or many prior trajectories.

We evaluate SKILLRE-
VISE in a unified verifier-driven harness across both standard and out-of-distribution skill-use settings. On SkillsBench (Li et al., 2026), SKILLRE-
VISE improves GPT-5.5 from 36.05% success without skills and 39.53% with one-shot skill generation to 61.63% after only

three revision rounds. To test whether these gains extend beyond the original benchmark distribution, we further adapt SkillLearnBench-Random and SWE-Skills-Bench-Hard into the same harness, and include an ALFWorld principle-absorption study for interactive embodied tasks. Across these benchmarks and multiple strong executors, including GPT-5.5, Qwen-3.6-Plus, and DeepSeek-V4-Pro, execution-grounded revision consistently improves skill performance, suggesting that SKILLRE-
VISE offers a robust and general approach to automated skill authoring.

This paper makes three contributions:

- We formulate cold-start skill improvement as bounded, execution-grounded revision of an existing LLM-authored skill, rather than retrieval or open-ended self-evolution from prior trajectories.
- We introduce SKILLRE-
VISE, which combines task-specific Diagnosis, reusable Principle Memory, and execution-anchored revision with verifier-first selection and utility fallback.
- We evaluate SKILLRE-
VISE across SkillsBench, SkillLearnBench-Random, SWE-Skills-Bench-Hard, and an ALFWorld study, showing consistent gains over no-skill and one-shot Skill Creator baselines across multiple executor models.

2 Related Work

Agent skills and skill benchmarks. Agent skills have emerged as reusable procedural artifacts for extending LLM agents beyond one-off prompts and atomic tools (Anthropic, 2025; Zhou et al., 2026; Liu et al., 2026b; Zhong et al., 2026; Chen et al., 2026). SkillsBench evaluates whether skills improve agents through diverse professional tasks and shows that curated skills can help whereas self-generated skills are unreliable (Li et al., 2026). SkillLearnBench studies continual learning methods for agent skill generation on real-world tasks, emphasizing whether agents can accumulate task experience into reusable skills over time (Zhong et al., 2026). WildSkills studies more realistic retrieval settings in which agents must select from large skill collections (Liu et al., 2026b), and SWE-Skills-Bench evaluates skill injection in software-engineering repositories with deterministic tests (Han et al., 2026). These studies motivate our setting: the value of a skill depends on whether it is aligned with the executor, task context, and verifier, not merely on whether the skill exists. However,

these benchmarks mainly evaluate whether skills help or can be learned from experience, rather than how an imperfect initial skill can be revised under bounded execution feedback.

Self-evolving skills. Recent systems learn or evolve reusable skills from experience. SkillX constructs hierarchical skill knowledge bases from trajectories (Wang et al., 2026); SkillRL builds a hierarchical SkillBank and retrieves general and task-specific heuristics for policy improvement (Xia et al., 2026); Skill-Pro learns executable procedural skills using a Skill-MDP and non-parametric PPO (Mi et al., 2026); MemSkill evolves memory operations as skills (Zhang et al., 2026); AutoSkill abstracts personalized skills from interaction histories (Yang et al., 2026); EvoSkill discovers and refines reusable skills through failure analysis (Alzubi et al., 2026); and SkillClaw evolves skills collectively from multi-user trajectories (Ma et al., 2026). However, these methods often require accumulated trajectories, repeated practice, or evolving skill libraries, making them less suitable for bounded cold-start revision.

Agent memory, reflection, and verifier-guided improvement. Our work is also related to agent memory and self-improvement methods that store experience for future use. Prior systems use episodic memory, semantic memory, retrieved reflections, or accumulated experience-skill knowledge to improve long-horizon interaction and planning (Packer et al., 2023; Zhong et al., 2024; Zhou et al., 2025; Jiang et al., 2026). Reflection- and critique-based methods derive textual feedback from failures or model outputs and feed it into subsequent attempts or training (Shinn et al., 2023; Yin et al., 2024; Zong et al., 2025). Program-repair and coding-agent systems similarly exploit execution feedback from tests or compilers to iteratively improve generated code (Xia et al., 2025; Xu et al., 2025). SKILLREVISE differs in its unit of improvement: rather than only revising a single trajectory or final answer, it revises a reusable skill artifact and validates whether that revision improves downstream execution under a fixed verifier.

3 Method: SKILLREVISE

This section describes SKILLREVISE as a bounded, execution-grounded revision loop for improving an existing LLM-authored skill. As shown in Figure 2, starting from an initial skill S_0 , the framework executes the skill, diagnoses verifier-facing failures,

retrieves and binds repair principles, edits with executable anchors, re-executes the candidate, and applies success-prioritized selection. We first define the two sources of revision information, Diagnosis and Principle Memory, then formalize the Revision Operator that converts them into concrete skill edits and the bounded episode that governs re-execution, early success selection, and utility fallback.

3.1 Diagnosis

Diagnosis is the task-specific component of SKILLREVISE. For the i -th revision attempt, we represent Diagnosis as

$$D_i = (V_i, A_i, \mathcal{K}_i), \quad (1)$$

where V_i is the *verification specification*, A_i is the *failure attribution*, and $\mathcal{K}_i$ is the *preservation constraints*. The verification specification V_i describes the observable requirements used to judge the run, including output paths, schemas, formats, terminal sentinels, and pass/fail assertions. The attribution component A_i summarizes failed checks, observed behavior, probable causes, and defect labels, indicating whether and how the current skill contributed to failure. The preservation constraints $\mathcal{K}_i$ record checks already satisfied and choices likely responsible for them. Together, D_i turns raw execution evidence into repair constraints: it specifies what to repair, what evidence supports the repair, and what behavior must remain intact.

3.2 Principle Memory

Principle Memory $\mathcal{M}$ stores reusable repair principles rather than task solutions. Each principle abstracts a recurring skill-design defect into an operational repair pattern. A principle entry specifies when the repair should be considered, what defect it addresses, how the skill should change, what executor action the edit should induce, how the repair should be verified, and when the principle should not transfer. Principle Memory is initialized with a set of seven repair principles. Appendix L lists these seed entries, and the appendix analyzes representative clean repair contrasts.

3.3 Revision Operator

The Revision Operator defines the local edit step in SKILLREVISE. Given the current skill S_i , Diagnosis D_i , and a set of bound principles P_i supplied by the revision episode, it maps them to a proposed revised skill and a revision trace:

$$(\hat{S}_{i+1}, z_i) = \mathcal{R}_\phi(S_i, D_i, P_i). \quad (2)$$

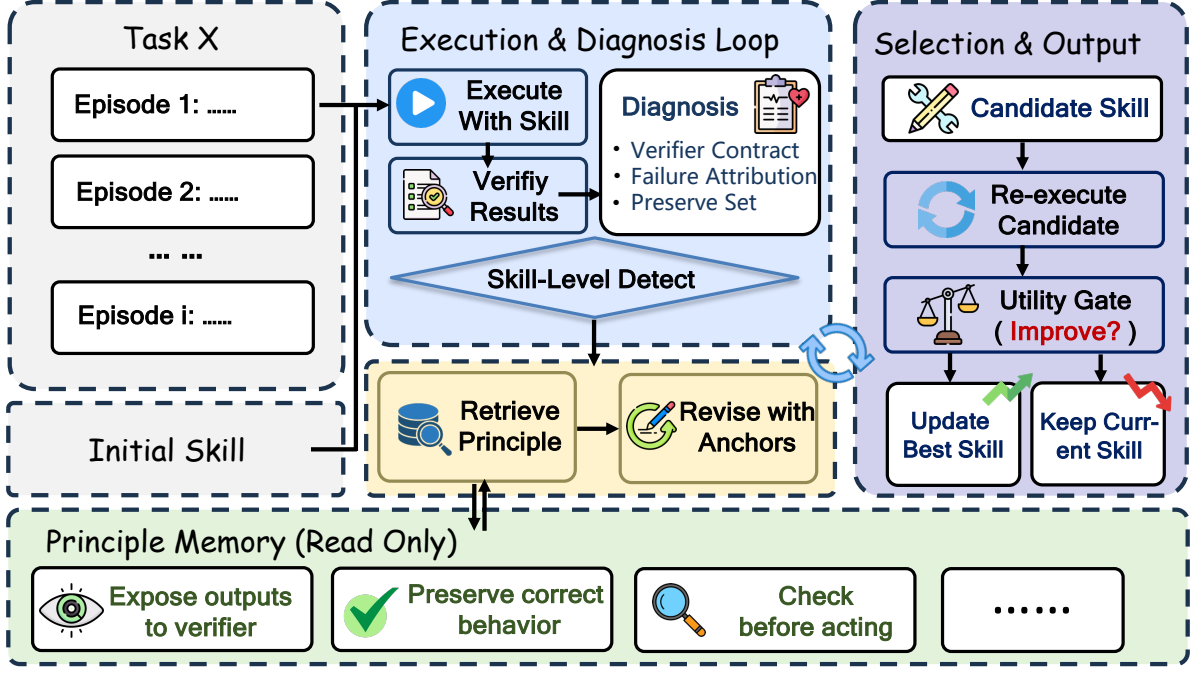


Figure 2: SKILLREVISE pipeline. Solid arrows show one bounded execution-grounded revision episode: execute the current skill, diagnose evidence, retrieve and bind active principles, generate an anchored candidate, re-execute it, and retain the first verifier-passing skill, with utility fallback only if no candidate succeeds. The dashed arrow denotes optional post-evaluation memory absorption.

The trace z_i documents the evidence-to-edit mapping behind each revision, including the applied principles, targeted spans in S_i , relevant verification and preservation constraints, expected acceptance signals, and execution anchors.

An *execution anchor* specifies how a textual edit should change executor behavior: the action to perform, the artifact or evidence to inspect, and where the instruction should appear in $\mathcal{I}(S_i)$. For example, an anchored JSON repair may require the executor to reload and parse the generated file, check required keys, and run local validation.

3.4 Bounded Revision Episode

A SKILLREVISE episode operationalizes the interfaces above under a finite revision budget. The budget constrains how many repair attempts may be evaluated, but it does not require the final generated candidate to be returned. Given a task T , an initial skill S_0 , an executor policy π_θ , a fixed Principle Memory $\mathcal{M}$, and a revision budget B , the episode generates candidate revisions and returns the first evaluated skill that passes the verifier. If no evaluated skill succeeds within the budget, it returns a utility fallback. Unless otherwise stated, the standard setting uses $B = 3$ revision rounds; the execution protocol, pseudocode, and fixed implementation settings appear in Appendices B and C.

Episode state. The episode maintains a current skill S_i , a utility fallback skill S_{fb} , and an observed candidate set $\mathcal{H}$. Initially, $S_i = S_0$, $S_{fb} = S_0$, and $\mathcal{H} = \{S_0\}$. At each round, SKILLREVISE executes the current skill, diagnoses the resulting evidence, generates diagnosis-gated revisions when warranted, and re-executes the candidate. If a candidate passes the verifier, it is selected and the episode stops; otherwise, S_{fb} is updated only when measured utility improves.

The episode separates the search base from the returned artifact. If budget remains after generating $\hat{S}_{i+1}$, the next round sets $S_{i+1} \leftarrow \hat{S}_{i+1}$ so later diagnoses inspect the edited artifact. The returned artifact is instead the first verifier-passing skill, or the utility fallback if none succeeds.

Round transition. At round i , execution of the current skill produces

$$e_i = (\tau_i, v_i, r_i, c_i) = \Phi(T, S_i, \pi_\theta), \quad (3)$$

where τ_i is the trajectory, v_i is verifier feedback or evaluation evidence, r_i is the outcome score or pass/fail reward, and c_i records costs such as tokens, tool calls, steps, and latency.

SKILLREVISE constructs Diagnosis D_i from e_i and uses it as a gate for revision. If A_i does not support a skill-level defect, the episode abstains

from editing the skill for that evidence. When revision is warranted, SKILLREVISE retrieves a top- m candidate set from Principle Memory:

$$q_i = Q(T, D_i), \quad C_i = \text{Retrieve}_m(q_i, \mathcal{M}). \quad (4)$$

The retrieval query uses task metadata and Diagnosis, including task family, acceptance criteria, diagnosis labels, causal judgment, rewrite targets, and evidence snippets. Hybrid retrieval combines sparse matching, which captures explicit labels, verifier terms, and rewrite targets, with dense matching, which captures semantically similar repair situations. Their rankings are combined by weighted reciprocal-rank fusion:

$$\text{score}(p) = \frac{w_s}{\kappa + \text{rank}_s(p)} + \frac{w_d}{\kappa + \text{rank}_d(p)}, \quad (5)$$

where m is the retrieval width, distinct from the revision budget B , and κ is the fusion constant.

Retrieval produces candidate principles, not final repair instructions. SKILLREVISE binds retrieved principles to the current diagnosis by retaining only those whose evidence requirements are satisfied and whose transfer constraints are not violated:

$$P_i = \text{Bind}(C_i, D_i). \quad (6)$$

The episode then calls the Revision Operator in Eq. 2 to obtain a candidate revision $\hat{S}_{i+1}$ and trace z_i . The candidate is re-executed on the same task and added to the observed candidate set before success-prioritized selection is applied.

Success-prioritized selection. Selection prioritizes verifier success before utility. Let $\hat{\mathcal{S}}_{\leq B}$ denote all generated and evaluated revision candidates within budget B . The observed set is

$$\mathcal{H}_{\leq B} = \{S_0\} \cup \hat{\mathcal{S}}_{\leq B}, \quad (7)$$

and let $\mathcal{P}_{\leq B} = \{S \in \mathcal{H}_{\leq B} : \text{succ}(S, T) = 1\}$ denote the verifier-passing candidates. The selected artifact $S_{\leq B}^*$ is defined as

$$S_{\leq B}^* = \begin{cases} \arg \min_{S \in \mathcal{P}_{\leq B}} \text{idx}(S), & \mathcal{P}_{\leq B} \neq \emptyset, \\ \arg \max_{S \in \mathcal{H}_{\leq B}} U(S, T), & \mathcal{P}_{\leq B} = \emptyset, \end{cases} \quad (8)$$

where $\text{idx}(S_j) = j$ is the evaluated version index. Thus, a successful v_0 is kept, the first successful revision is adopted when v_0 fails, and utility is used only to choose among failed candidates when no success is found. Increasing the revision budget

expands the candidate set but does not imply that the final generated revision is selected. For the fallback case and auxiliary analysis, the utility may combine measured outcome, success-conditioned efficiency, transfer, and interference cost:

$$\begin{aligned} U(S, T) = & \alpha \Delta_{\text{succ}}(S, T) \\ & + \beta g_{\text{succ}}(S, T) \Delta_{\text{eff}}(S, T) \\ & + \gamma \Delta_{\text{trans}}(S, \mathcal{F}) \\ & - \lambda C_{\text{intf}}(S, \mathcal{F}). \end{aligned} \quad (9)$$

Here, Δ_{succ} , Δ_{eff} , and Δ_{trans} denote changes in task outcome or success, execution efficiency, and transfer performance; C_{intf} measures interference on other tasks; and $\alpha, \beta, \gamma, \lambda$ weight the corresponding terms. The gate $g_{\text{succ}}(S, T)$ conditions efficiency credit on successful execution. For binary verifiers, we set $g_{\text{succ}}(S, T) = \mathbf{1}[\text{succ}(S, T) = 1]$, so lower-cost candidates are rewarded only after satisfying the verifier. This prevents the utility from favoring revisions that appear efficient simply because they stop early or omit required work.

Optional memory absorption. The dashed arrow in Figure 2 denotes optional post-evaluation or deployment-time memory maintenance. Reported benchmark results use the fixed Principle Memory available before evaluation; online absorption is not used to improve later test tasks unless explicitly stated. When enabled, absorption is conservative: candidate principles must be evidence-backed, skill-level, utility-improving, and transferable, and filters remove task-specific constants, output paths, hidden answers, and verifier-specific shortcuts. Appendix K lists the prompt interfaces.

4 Experiments

4.1 Experimental Setup

We evaluate SKILLREVISE as an execution-grounded skill revision framework on three verifier-driven benchmarks: the original SkillsBench, a 50-task SkillLearnBench-Random suite, and a 70-task SWE-Skills-Bench-Hard suite. Each benchmark uses a revision budget of three. Unless otherwise stated, skill authoring and revision use GPT-5.5, while task execution is evaluated across GPT-5.5 (OpenAI, 2025), Opus-4.7 (Anthropic, 2026), Kimi-2.6 (Kimi Team, 2025), Qwen-3.6-Plus (Qwen Team, 2025), and DeepSeek-V4-Pro (DeepSeek-AI, 2026). We additionally report an ALFWorld (Shridhar et al., 2020) principle-absorption study in Section 5.4.

Model	Method	SkillsBench		SkillLearn-R		SWE-Skills-H		Overall
		Succ.	Δpp	Succ.	Δpp	Succ.	Δpp	
GPT-5.5	No skill	31/86	—	20/50	—	28/70	—	79/206
	Skill-Creator	34/86	+3.49	17/50	-6.00	27/70	-1.43	78/206
	Skill v_0	35/86	+4.65	23/50	+6.00	29/70	+1.43	87/206
	Rev. v_1	47/86	+18.60	29/50	+18.00	30/70	+2.86	106/206
	Rev. v_3	53/86	+25.58	29/50	+18.00	33/70	+7.14	115/206
Opus-4.7	No skill	16/86	—	7/50	—	32/70	—	55/206
	Skill-Creator	28/86	+13.95	7/50	+0.00	25/70	-10.00	60/206
	Skill v_0	17/86	+1.16	7/50	+0.00	23/70	-12.86	47/206
	Rev. v_1	32/86	+18.60	20/50	+26.00	29/70	-4.29	81/206
	Rev. v_3	44/86	+32.56	25/50	+36.00	31/70	-1.43	100/206
Kimi-2.6	No skill	16/86	—	6/50	—	25/70	—	47/206
	Skill-Creator	17/86	+1.16	10/50	+8.00	20/70	-7.14	47/206
	Skill v_0	19/86	+3.49	11/50	+10.00	23/70	-2.86	53/206
	Rev. v_1	28/86	+13.95	15/50	+18.00	25/70	+0.00	68/206
	Rev. v_3	37/86	+24.42	22/50	+32.00	27/70	+2.86	86/206
Qwen-3.6-Plus	No skill	6/86	—	5/50	—	22/70	—	33/206
	Skill-Creator	8/86	+2.33	8/50	+6.00	24/70	+2.86	40/206
	Skill v_0	9/86	+3.49	7/50	+4.00	26/70	+5.71	42/206
	Rev. v_1	24/86	+20.93	12/50	+14.00	34/70	+17.14	70/206
	Rev. v_3	27/86	+24.42	15/50	+20.00	35/70	+18.57	77/206
DeepSeek-V4-Pro	No skill	14/86	—	12/50	—	23/70	—	49/206
	Skill-Creator	21/86	+8.14	16/50	+8.00	25/70	+2.86	62/206
	Skill v_0	23/86	+10.47	14/50	+4.00	28/70	+7.14	65/206
	Rev. v_1	34/86	+23.26	23/50	+22.00	29/70	+8.57	86/206
	Rev. v_3	41/86	+31.40	24/50	+24.00	30/70	+10.00	95/206

Table 1: Main results. Each benchmark cell reports successes and Δpp over no skill; Overall sums successes across the three benchmarks (206 tasks). SkillLearn-R denotes SkillLearnBench-Random; SWE-Skills-H denotes SWE-Skills-Bench-Hard. GPT-5.5 uses the Codex harness; other executors use Claude Code. Shading begins at the Method column and marks Rev. v_3 ; bold marks Rev. v_3 beating both baselines.

Evaluation conditions. We compare five evaluation conditions under the same task interface, executor, workspace, and verifier, so differences come from the installed skill artifact and revision budget. This controlled comparison isolates the effect of the installed skill artifact: no guidance, one-shot skill authoring, or execution-grounded revision. *No skill* runs the executor without any skill artifact. *Skill-Creator* installs a one-shot LLM-authored skill but performs no execution-grounded revision. *Skill v_0* is the initial skill produced by the SKILLREVISE authoring stage before observing verifier feedback, diagnosis, or revision. These first three conditions define the pre-revision baselines. *Revision v_1* and *Revision v_3* denote budgeted revision conditions with one and three allowed repair rounds, respectively. For each condition, we report the first verifier-passing evaluated candidate if one exists, and otherwise the utility fallback. Thus, v_1/v_3 are budget-selected outcomes, not necessarily the most recently generated revision.

Benchmark protocol. We repackage SWE-Skills-Bench-Hard (Han et al., 2026) and SkillLearnBench-Random (Zhong et al., 2026) into the same task-directory interface used by SkillsBench. For both adapted benchmarks, we preserve the original task instructions, environments, and test-based evaluators where applicable. Task success is the primary metric. We also report outcome score, utility, tokens, tool calls, steps, and latency. Unless explicitly stated, reported benchmark results use a fixed Principle Memory available before evaluation, and do not use online absorption from test tasks. Appendices A and I give task-bundle records and the retained-skill leakage boundary.

ALFWorld setup. For ALFWorld, we use 25 calibration tasks to absorb a compact 10-principle bank, then evaluate on 100 valid-seen and valid-unseen tasks. Qwen3-8B is the executor. The initial skill v_0 is evaluated without the bank; later revisions retrieve from it. Evaluation tasks remain held out from online absorption.

4.2 Main Results

SKILLREVISE delivers broad gains over both no-skill execution and one-shot skill generation. Table 1 compares no-skill execution, one-shot Skill-Creator skills, the initial v_0 skill, and selected revised skills after one and three revision rounds. Across the 206 evaluated tasks, Revision v3 consistently improves overall success relative to no-skill execution: from 79 to 115 for GPT-5.5, 55 to 100 for Opus-4.7, 47 to 86 for Kimi-2.6, 33 to 77 for Qwen-3.6-Plus, and 49 to 95 for DeepSeek-V4-Pro. For the top executor, GPT-5.5, the gain spans all three benchmarks: Revision v3 raises SkillsBench success from 31/86 to 53/86, SkillLearnBench-Random from 20/50 to 29/50, and SWE-Skills-Bench-Hard from 28/70 to 33/70. The effect is not confined to GPT-5.5: Kimi-2.6, Qwen-3.6-Plus, and DeepSeek-V4-Pro improve over no skill on all three benchmarks, while Opus-4.7 achieves large gains on SkillsBench and SkillLearnBench-Random and remains close to no-skill performance on SWE-Skills-Bench-Hard.

One-shot skill generation is not a reliable substitute for revision. Skill-Creator sometimes helps, but its effect is inconsistent: it is slightly worse than no skill overall for GPT-5.5 (78/206 versus 79/206), tied overall for Kimi-2.6, and below no skill on several benchmark cells, including GPT-5.5 SkillLearnBench-Random and SWE-Skills-Bench-Hard. The initial v_0 skill is also not enough by itself; for example, Opus-4.7 drops from 55/206 without skills to 47/206 with v_0 . In contrast, Revision v3 is the strongest overall condition for every executor in Table 1. This gap indicates that the improvement comes from diagnosing failures with execution evidence, re-executing revised candidates, and retaining verifier-passing skills, not merely from giving the agent any skill file.

Most recovery appears early, while later rounds expand the candidate pool. Revision v1 already improves over v_0 in every model-benchmark setting in Table 1. On SkillsBench, GPT-5.5 rises from 40.7% to 54.7%, Opus-4.7 from 19.8% to 37.2%, Kimi-2.6 from 22.1% to 32.6%, Qwen-3.6-Plus from 10.5% to 27.9%, and DeepSeek-V4-Pro from 26.7% to 39.5%. Later rounds still matter because unresolved tasks receive additional verifier-tested candidates; for example, Opus-4.7 rises from 37.2% to 51.2% on SkillsBench and from 40.0% to 50.0% on SkillLearnBench-Random. The smaller gains on

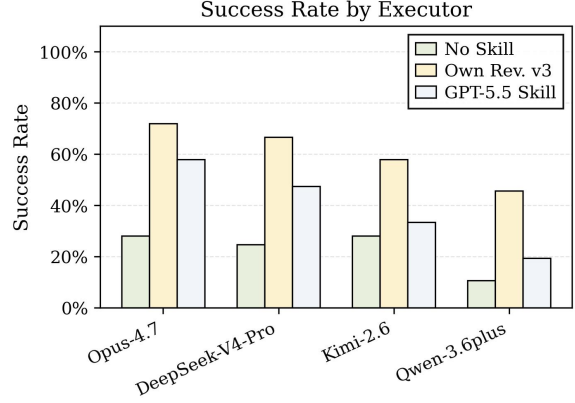


Figure 3: Cross-model transfer on the 57-task GPT-5.5 source-success subset. Each group reports no-skill execution, executor-specific Revision v3, and fixed GPT-5.5-produced skills.

SWE-Skills-Bench-Hard suggest that repository-specific, long-horizon failures are harder to repair through generic skill revision. Appendix E reports a domain-level SkillsBench breakdown, and Table 5 reports GPT-5.5 cost.

5 Analysis and Discussion

5.1 Analysis of Skills Transferability

This analysis aims to examine whether revised skills encode transferable procedural knowledge, rather than serving as cue-based output of the executor model during revision. On the 57-task GPT-5.5 source-success subset, we compare three conditions for the four target executors with verified transfer runs: no-skill execution, the executor’s own Revision v3 skill, and the same fixed GPT-5.5-produced skill artifact. Figure 3 shows the four-executor comparison, and Appendix D reports the corresponding success counts.

The transfer results show partial but meaningful cross-executor portability. Across all four target executors, the fixed GPT-5.5-produced skill improves over no-skill execution, indicating that revision can produce procedural constraints and verifier-facing checks that are not tied exclusively to the source model. At the same time, transfer is not uniformly beneficial: executor-specific Revision v3 is typically stronger than direct transfer, and some executors benefit less from the GPT-5.5 skill. This pattern suggests that revised skills include both reusable task execution paths and prompt conventions tailored to specific execution agents. The former reveals the generalizability of skill generation, while the latter explains why executor-aware revision remains important.

Budget	v_0	v_1	v_2	v_3	v_4	v_5
maxrev1	55	31	—	—	—	—
maxrev2	46	23	17	—	—	—
maxrev3	42	19	13	12	—	—
maxrev4	38	20	12	12	4	—
maxrev5	37	20	11	11	4	3

Table 2: Selected skill versions by revision budget.

Method	Succ.	Impact
No ablation	53/86	0.00%
w/o Diagnosis	28/86	↓ 29.07%
w/o Preserve Ledger	42/86	↓ 12.79%
w/o Execution Anchors	44/86	↓ 10.47%
w/o Principles	45/86	↓ 9.30%
Free-form Revision	52/86	↓ 1.16%

Table 3: Ablation results for SkillsBench. Impact is the absolute success-rate drop from no ablation.

5.2 Revision Rounds and Selection

Revision rounds affect SKILLREVISE through both candidate generation and final selection. Here we focus on the selection behavior, while Appendix B.3 reports the full success-rate and score sweep across revision caps. The budget maxrev k allows up to k attempted repairs, but the retained skill is still chosen by success-prioritized selection with utility fallback. Increasing the budget therefore does not mean that the newest rewrite should automatically be deployed; it only expands the set of evaluated candidates available to the selector.

Table 2 shows this effect directly. With maxrev1, the selected artifact must come from either the initial v_0 skill or the first revision v_1 . As the budget increases, later versions become available for tasks that remain unresolved, but early skills continue to dominate the retained set: even under maxrev5, the selector keeps v_0 for 37 tasks and v_1 for 20 tasks, while v_4 and v_5 account for only a small tail. This pattern supports the intended role of revision rounds as bounded search rather than blind iterative rewriting: later rounds add opportunities, while selection preserves earlier skills when they already pass the verifier or remain empirically stronger.

5.3 Component Ablations

To verify whether the effectiveness of SKILLREVISE stems from its structured revision framework, rather than simply from the model being given additional opportunities to rewrite skills, we compare exactly six methods: no ablation, Free-form Revision, w/o Diagnosis, w/o Principles, w/o Execution Anchors, and w/o Preserve Ledger. Each controlled

Method	SR	Setting / note
Reflexion (Shinn et al., 2023)	97.0	ReAct + reflection; multiple trials
ReflAct (Kim et al., 2025)	93.3	goal-state reflection agent
GiGPO (Feng et al., 2025)	90.8	Qwen2.5-7B RL post-training
Const.-context skills	89.6	Qwen3-8B SFT+RL skill modules
GRPO (DeepSeek-AI, 2024)	77.6	Qwen2.5-7B RL post-training
SKILLREVISE	71.0	Qwen3-8B; 25-task absorption set
SimpleMem (Liu et al., 2026a)+GRPO	62.5	memory + RL baseline
Mem0 (Chhikara et al., 2025)+GRPO	54.7	memory + RL baseline
ExpeL (Zhao et al., 2024)	46.3	experience-to-insight baseline
EvolveR (Wu et al., 2025)	43.8	self-evolving memory baseline
MemP (Fang et al., 2025)	41.4	procedural memory baseline
Mem0 (Chhikara et al., 2025)	33.6	external memory baseline

Table 4: Representative ALFWorld results under 25-task principle absorption and maxrev3 evaluation.

variant reuses the same initial skill S_0 , executor, verifier, and maxrev3 budget.

Table 3 shows that Diagnosis is the dominant component: removing it drops success from 53/86 to 28/86, indicating that revision needs an explicit bridge from verifier evidence to skill-level defects. The preserve ledger, execution anchors, and Principle Memory each account for substantial but smaller drops, suggesting that SKILLREVISE benefits from both regression control and reusable repair patterns. Free-form Revision remains close in raw success, indicating that the structured output format mainly serves to make revisions grounded, attributable, and inspectable rather than simply enabling rewrites; Appendices F and H provide the corresponding traces and failure-mode analysis.

5.4 ALFWorld Principle Absorption

To test Principle Memory beyond verifier-driven file tasks, we run ALFWorld with Qwen3-8B. Agents issue text commands rather than edit files or repositories, so success depends on admissible action syntax and state tracking. From 25 calibration tasks, we absorb a compact 10-principle bank covering recurring failures such as premature stopping, invalid actions, budget loss, and two-object goals, then evaluate on 100 valid-seen/unseen tasks.

The selected run solves 71/100 tasks, with 33/44 valid-seen and 38/56 valid-unseen successes. Table 11 in Appendix G breaks this down by task family: success is highest on heat/place and light-inspection tasks, while clean/place remains hardest. Table 4 contextualizes the result: SKILLREVISE outperforms memory-only baselines but remains below trained or multi-trial systems. The selector keeps v_0 , v_1 , and v_2 for 62, 31, and 7 tasks, respectively. Thus, the absorbed bank acts as targeted repair guidance rather than a default trigger, with principles used selectively rather than forcing revision on every episode.

6 Conclusion

SKILLREVISE treats skills as execution-grounded artifacts that can be diagnosed, revised, and selected with verifier feedback. Across SkillsBench, SkillLearnBench-Random, and SWE-Skills-Bench-Hard, this bounded revision loop improves over both no-skill execution and one-shot Skill Creator in most evaluated settings. The results suggest that reusable agent skills are most effective when they are not static advice, but testable procedural memory refined by observed failures.

Limitations

SKILLREVISE depends on verifier-visible feedback and spends additional execution budget to obtain it. Sparse or misaligned tests can lead revision to overfit visible checks, and some tasks remain better solved by direct no-skill exploration because maxrev3 does not fall back to no-skill execution. Our evaluation is also limited to verifier-based benchmarks rather than long-running or safety-critical deployments. Appendix J discusses these limitations in more detail.

References

- Salaheddin Alzubi, Noah Provenzano, Jaydon Bingham, Weiyuan Chen, and Tu Vu. 2026. *Evoskill: Automated skill discovery for multi-agent systems*. *Preprint*, arXiv:2603.02766.
- Anthropic. 2025. Agent skills. <https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview>. Technical documentation. Accessed 2026-05-20.
- Anthropic. 2026. *Claude opus 4.7 system card*. Model system card. Accessed 2026-05-26.
- Shiqi Chen, Jingze Gai, Ruochen Zhou, Jinghan Zhang, Tongyao Zhu, Junlong Li, Kangrui Wang, Zihan Wang, Zhengyu Chen, Klara Kaleb, Yihong Chen, Tianyi Li, Jiuding Li, Yu Su, Wenlong Huang, and Kaijie Zhu. 2026. *Skillcraft: Can llm agents learn to use tools skillfully?* *Preprint*, arXiv:2603.00718.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. *Mem0: Building production-ready ai agents with scalable long-term memory*. *Preprint*, arXiv:2504.19413.
- DeepSeek-AI. 2024. *Deepseek-v3 technical report*. *Preprint*, arXiv:2412.19437.
- DeepSeek-AI. 2026. *Deepseek-v4: Towards highly efficient million-token context intelligence*. Technical report.
- Runnan Fang, Yuan Liang, Xiaobin Wang, Jialong Wu, Shuofei Qiao, Pengjun Xie, Fei Huang, Huajun Chen, and Ningyu Zhang. 2025. *Memp: Exploring agent procedural memory*. *Preprint*, arXiv:2508.06433.
- Lang Feng, Zhenghai Xue, Tingcong Liu, and Bo An. 2025. *Group-in-group policy optimization for llm agent training*. *Preprint*, arXiv:2505.10978.
- Yudong Gao, Zongjie Li, Yuanyuanyuan, Zimo Ji, Pingchuan Ma, and Shuai Wang. 2026. *Skillreducer: Optimizing llm agent skills for token efficiency*. *Preprint*, arXiv:2603.29919.
- Tingxu Han, Yi Zhang, Wei Song, Chunrong Fang, Zhenyu Chen, Youcheng Sun, and Lijie Hu. 2026. *Swe-skills-bench: Do agent skills actually help in real-world software engineering?* *Preprint*, arXiv:2603.15401.
- Guanyu Jiang, Zhaochen Su, Xiaoye Qu, and Yi R. Fung. 2026. *Xskill: Continual learning from experience and skills in multimodal agents*. *Preprint*, arXiv:2603.12056.
- Jeonghye Kim, Sojeong Rhee, Minbeom Kim, Dohyung Kim, Sangmook Lee, Youngchul Sung, and Kyomin Jung. 2025. *Reflact: World-grounded decision making in llm agents via goal-state reflection*. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 33433–33465.
- Kimi Team. 2025. *Kimi k2: Open agentic intelligence*. *Preprint*, arXiv:2507.20534.
- Junlong Li, Wenshuo Zhao, Jian Zhao, Weihao Zeng, Haoze Wu, Xiaochen Wang, Rui Ge, Yuxuan Cao, Yuzhen Huang, Wei Liu, Junteng Liu, Zhaochen Su, Yiyang Guo, Fan Zhou, Lueyang Zhang, Juan Micheli, Xingyao Wang, Xiang Yue, Shuyan Zhou, et al. 2025. *The tool decathlon: Benchmarking language agents for diverse, realistic, and long-horizon task execution*. *Preprint*, arXiv:2510.25726.
- Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, Shuyi Wang, Binxu Li, Qunhong Zeng, Di Wang, Xuandong Zhao, Yuanli Wang, Roey Ben Chaim, Zonglin Di, Yipeng Gao, et al. 2026. *Skillsbench: Benchmarking how well agent skills work across diverse tasks*. *Preprint*, arXiv:2602.12670.
- Jiaqi Liu, Yaofeng Su, Peng Xia, Siwei Han, Zeyu Zheng, Cihang Xie, Mingyu Ding, and Huaxiu Yao. 2026a. *Simplemem: Efficient lifelong memory for llm agents*. *Preprint*, arXiv:2601.02553.
- Yujian Liu, Jiabao Ji, Li An, Tommi Jaakkola, Yang Zhang, and Shiyu Chang. 2026b. *How well do agentic skills work in the wild: Benchmarking llm skill usage in realistic settings*. *Preprint*, arXiv:2604.04323.
- Ziyu Ma, Shidong Yang, Yuxiang Ji, Xucong Wang, Yong Wang, Yiming Hu, Tongwen Huang, and Xiangxiang Chu. 2026. *Skillclaw: Let skills*

- evolve collectively with agentic evolver. *Preprint*, arXiv:2604.08377.
- Qirui Mi, Zhijian Ma, Mengyue Yang, Haoxuan Li, Yisen Wang, Haifeng Zhang, and Jun Wang. 2026. [Skill-pro: Learning reusable skills from experience via non-parametric ppo for llm agents](#). *Preprint*, arXiv:2602.01869.
- OpenAI. 2025. [OpenAI GPT-5 system card](#). *Preprint*, arXiv:2601.03267.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2023. [Memgpt: Towards llms as operating systems](#). *Preprint*, arXiv:2310.08560.
- Qwen Team. 2025. [Qwen3 technical report](#). *Preprint*, arXiv:2505.09388.
- Xiyu Ren, Zhaowei Wang, Yiming Du, Zhongwei Xie, Chi Liu, Xinlin Yang, Haoyue Feng, Wenjun Pan, Tianshi Zheng, Baixuan Xu, Yuxin Jiang, Qiqi Wang, Haoran Li, and Yangqiu Song. 2026. [Memlens: Benchmarking multimodal long-term memory in large vision-language models](#). *Preprint*, arXiv:2605.14906.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. [Reflexion: Language agents with verbal reinforcement learning](#). *Advances in Neural Information Processing Systems*, 36:8634–8652.
- Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. 2020. [Alfworld: Aligning text and embodied environments for interactive learning](#). *Preprint*, arXiv:2010.03768.
- Zhaochen Su, Jincheng Gao, Hangyu Guo, Zhenhua Liu, Lueyang Zhang, Xinyu Geng, Shijue Huang, Peng Xia, Guanyu Jiang, Cheng Wang, Yue Zhang, Yi R. Fung, and Junxian He. 2026. [Agentvista: Evaluating multimodal agents in ultra-challenging realistic visual scenarios](#). *Preprint*, arXiv:2602.23166.
- Chenxi Wang, Zhuoyun Yu, Xin Xie, Wuguannan Yao, Runnan Fang, Shuofei Qiao, Kexin Cao, Guozhou Zheng, Xiang Qi, Peng Zhang, and Shumin Deng. 2026. [Skillx: Automatically constructing skill knowledge bases for agents](#). *Preprint*, arXiv:2604.04804.
- Rong Wu, Xiaoman Wang, Jianbiao Mei, Pinlong Cai, Daocheng Fu, Cheng Yang, Licheng Wen, Xuemeng Yang, Yufan Shen, Yuxin Wang, Shichao Dong, Xiangu Wang, Jiwei Zhang, Zhe Zhang, Shenglai Zeng, Yu Qiao, Peng Gao, Kaipeng Wang, Ji-Rong Wen, et al. 2025. [Evolver: Self-evolving llm agents through an experience-driven lifecycle](#). *Preprint*, arXiv:2510.16079.
- Chunqiu Steven Xia, Zhe Wang, Yan Yang, Yuxiang Wei, and Lingming Zhang. 2025. [Live-swe-agent: Can software engineering agents self-evolve on the fly?](#) *Preprint*, arXiv:2511.13646.
- Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, Zeyu Zheng, Cihang Xie, and Huaxiu Yao. 2026. [Skillrl: Evolving agents via recursive skill-augmented reinforcement learning](#). *Preprint*, arXiv:2602.08234.
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. [A-mem: Agentic memory for llm agents](#). *Preprint*, arXiv:2502.12110.
- Yutao Yang, Junsong Li, Qianjun Pan, Bihao Zhan, Yuxuan Cai, Lin Du, Jie Zhou, Kai Chen, Qin Chen, Xin Li, Bo Zhang, and Liang He. 2026. [Autoskill: Experience-driven lifelong learning via skill self-evolution](#). *Preprint*, arXiv:2603.01145.
- Xunjian Yin, Xinyi Wang, Liangming Pan, Li Lin, Xiaojun Wan, and William Yang Wang. 2024. [Gödel agent: A self-referential agent framework for recursive self-improvement](#). *Preprint*, arXiv:2410.04444.
- Haozhen Zhang, Quanyu Long, Jianzhu Bao, Tao Feng, Weizhi Zhang, Haodong Yue, and Wenya Wang. 2026. [Memskill: Learning and evolving memory skills for self-evolving agents](#). *Preprint*, arXiv:2602.02474.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. [Expel: Llm agents are experiential learners](#). In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19632–19642.
- Shanshan Zhong, Yi Lu, Jingjie Ning, Yibing Wan, Lihan Feng, Yuyi Ao, Leonardo F. R. Ribeiro, Markus Dreyer, Sean Ammirati, and Chenyan Xiong. 2026. [Skilllearnbench: Benchmarking continual learning methods for agent skill generation on real-world tasks](#). *Preprint*, arXiv:2604.20087.
- Wanjuan Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2024. [Memorybank: Enhancing large language models with long-term memory](#). In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19724–19731.
- Yingli Zhou, Wang Shu, Yaodong Su, Wenchuan Du, Yixiang Fang, and Xuemin Lin. 2026. [A comprehensive survey on agent skills: Taxonomy, techniques, and applications](#). *Preprint*, arXiv:2605.07358.
- Zijian Zhou, Ao Qu, Zhaoxuan Wu, Sunghwan Kim, Alok Prakash, Daniela Rus, Jinhua Zhao, Bryan Kian Hsiang Low, and Paul Pu Liang. 2025. [Mem1: Learning to synergize memory and reasoning for efficient long-horizon agents](#). *Preprint*, arXiv:2506.15841.
- Qing Zong, Jiayu Liu, Tianshi Zheng, Chunyang Li, Baixuan Xu, Haochen Shi, Weiqi Wang, Zhaowei Wang, Chunkit Chan, and Yangqiu Song. 2025. [CritiCal: Can critique help LLM uncertainty or confidence calibration?](#) *Preprint*, arXiv:2510.24505.

A Benchmark Selection and Adaptation

A.1 Shared Bundle Interface

SkillsBench is already close to the file-based task interface used by our harness, while SkillLearnBench-Random and SWE-Skills-Bench-Hard require adaptation from their native formats. We therefore convert all three sources into the same SkillsBench-style outer interface before running any skill condition. Each selected task directory contains `instruction.md`, `task.toml`, a verifier entrypoint under `tests/`, and task assets under `environment/` when needed. The released manifests record the final 86 SkillsBench, 50 SkillLearnBench-Random, and 70 SWE-Skills-Bench-Hard tasks. For the two adapted benchmarks, we first export benchmark-native tasks into this shared layout and then apply an explicit task-selection file; the runner reads the resulting manifest rather than relying on an implicit hard-coded subset.

A.2 SkillsBench

The SkillsBench bundle contains the 86 selected tasks used for the main SkillsBench rows. We keep the task-facing instruction, environment files, and verifier contract in the shared bundle format, so the same runner can install no skill, a one-shot Skill Creator artifact, or a revised SKILLREVISE artifact before invoking the verifier.

A.3 SkillLearnBench-Random

SkillLearnBench-Random (Zhong et al., 2026) is originally organized around task families with multiple instances and benchmark-specific evaluation scripts. We use a representative 50-task random subset that spans the main SkillLearnBench task families rather than concentrating on a single task type. We flatten the selected instances into independent task directories by copying each instance’s instruction, metadata, environment assets, and tests into the shared bundle layout. The final subset is materialized from an explicit selection file, producing the manifest used for evaluation. We disable sibling-instance transfer context so that each selected instance is evaluated as a standalone task, and we score only verifier task success. The original SkillLearnBench coverage, safety, and executability LLM-judge metrics are not used in these comparisons, and oracle solution directories are not part of the public task bundle used for revision.

A.4 SWE-Skills-Bench-Hard

SWE-Skills-Bench-Hard (Han et al., 2026) is repository-based. Following the hard-setting motivation of SWE-Skills-Bench, we focus on tasks where direct LLM completion and self-generated skill use are weak, so the subset tests whether execution-grounded revision can improve skills in cases where simply adding a generated skill is not already sufficient. The final 70-task hard subset covers seven software-engineering skill families. The adaptation preserves the original software-engineering prompt, repository identity, configured environment, and task-specific tests while exposing them through the same task-directory interface. The exporter reads the benchmark configuration and task prompts, creates `instruction.md` and `task.toml`, builds the task environment from the original base image and repository commit, and copies the corresponding test files into the task verifier directory. Reference skills from the source benchmark are not exposed as solution artifacts during SKILLREVISE; the evaluated agent sees the task prompt, workspace, and verifier feedback under the same conditions as the other verifier-based benchmarks.

A.5 ALFWorld

ALFWorld is not converted into the file-based task bundle above. It is used as an interactive environment for the principle-absorption study in Section 5.4: a separate calibration split produces a compact principle bank, and the 100-task evaluation set is then executed without online absorption from evaluation tasks.

B Execution Protocol and Algorithm

B.1 Shared Evaluation Protocol

For SkillsBench, SkillLearnBench-Random, and SWE-Skills-Bench-Hard, *no skill*, *Skill Creator*, and SKILLREVISE use the same task interface, executor, verifier, and result aggregation whenever a matched run is available. *No skill* executes without an installed skill. *Skill Creator* installs a one-shot LLM-authored skill and performs no execution-grounded revision. SKILLREVISE begins with its own initially authored skill v_0 , generated before any execution feedback, then uses a fixed Principle Memory and up to three revision rounds. It reports the first verifier-passing evaluated skill; if no evaluated skill succeeds, it reports the highest-utility fallback rather than the last generated candidate.

Test-task traces are not absorbed back into Principle Memory for later test tasks.

B.2 GPT-5.5 SkillsBench Execution Profile

For the GPT-5.5 SkillsBench rows, skill authoring, skill revision, and task execution all use GPT-5.5. The executor is the Codex ACP agent under the Docker BenchFlow backend, and every candidate is scored by the same task verifier. *No skill* runs one verifier-scored execution without an installed skill. *Skill Creator* makes one LLM-authored skill and runs one skill-conditioned execution. SKILL-REVISE first evaluates its initially authored v_0 skill, then attempts up to $B = 3$ execution-grounded revision rounds; each evaluated candidate is re-executed before it can be selected. The selector returns the earliest verifier-passing evaluated skill, or the highest-utility fallback when all evaluated skills fail, so a maxrev3 run may retain v_0, v_1, v_2 , or v_3 , and it does not fall back to the no-skill baseline. The final artifact records one adopted trace per task, with status, reward, return code, timeout flag, tokens, tool calls, steps, and latency. For the 86-task GPT-5.5 SkillsBench run, the selector retains v_0, v_1, v_2 , and v_3 on 43, 19, 12, and 12 tasks, respectively; the selected SKILLREVISE traces contain no timeout.

Cost view	Succ.	Revs	Tok.	Tools	Lat.
Direct no-skill execution	31/86	0.00	36.6k	21.0	492.4
Skill Creator	34/86	0.00	37.9k	20.9	329.7
Final selected skill execution	53/86	0.00	36.6k	21.7	335.0
Full maxrev3 selection	53/86	1.42	56.9k	33.6	484.5

Table 5: GPT-5.5 SkillsBench cost profile over 86 tasks. Final selected skill execution is the post-selection use cost; Full maxrev3 selection counts revision candidates from v_1 until first success or budget exhaustion, excluding v_0 . Tokens, tool calls, and latency are averaged per task.

B.3 Revision Budget Sweep

Table 6 reports the GPT-5.5 SkillsBench budget sweep that complements Section 5.2.

Budget	Succ.	Rate	Out.	Overall
maxrev1	47/86	54.65	0.577	0.138
maxrev2	51/86	59.30	0.617	0.183
maxrev3	53/86	61.63	0.644	0.213
maxrev4	55/86	63.95	0.658	0.235
maxrev5	56/86	65.12	0.669	0.248

Table 6: Effect of revision budget on SkillsBench.

B.4 Bounded Revision Algorithm

Algorithm 1 expands the bounded revision episode used in the main experiments. The standard setting uses $B = 3$ revision rounds, a fixed Principle Memory available before evaluation, and success-prioritized selection over evaluated skill candidates, with utility fallback only when no candidate succeeds. Online absorption from test tasks is disabled for the reported benchmark runs; the ALFWorld study is the only setting where a separate calibration split is used to build an absorbed principle bank before evaluation.

Algorithm 1 Bounded SKILLREVISE episode used for maxrev3

Require: task T , initial skill S_0 , executor π_θ , verifier Φ , fixed Principle Memory $\mathcal{M}$, budget $B = 3$

- 1: evaluate S_0 : $e_0 \leftarrow \Phi(T, S_0, \pi_\theta)$
- 2: initialize $S_{\text{cur}} \leftarrow S_0$, $S_{\text{fb}} \leftarrow S_0$, and $\mathcal{H} \leftarrow \{(S_0, e_0)\}$
- 3: **if** $\text{succ}(e_0)$ **then**
- 4: **return** S_0 and the full candidate history $\mathcal{H}$
- 5: **end if**
- 6: **for** $i = 0$ to $B - 1$ **do**
- 7: construct Diagnosis D_i from the current evidence e_i
- 8: **if** D_i does not support a skill-level defect **then**
- 9: record an abstention trace and stop revising this task
- 10: **break**
- 11: **end if**
- 12: retrieve candidates $C_i \leftarrow \text{Retrieve}_m(Q(T, D_i), \mathcal{M})$
- 13: bind principles $P_i \leftarrow \text{Bind}(C_i, D_i)$ using evidence requirements and transfer constraints
- 14: generate $(\hat{S}_{i+1}, z_i) \leftarrow \mathcal{R}_\phi(S_{\text{cur}}, D_i, P_i)$
- 15: re-execute the candidate: $\hat{e}_{i+1} \leftarrow \Phi(T, \hat{S}_{i+1}, \pi_\theta)$
- 16: append $(\hat{S}_{i+1}, \hat{e}_{i+1}, z_i)$ to $\mathcal{H}$
- 17: **if** $\text{succ}(\hat{e}_{i+1})$ **then**
- 18: **return** $\hat{S}_{i+1}$ and the full candidate history $\mathcal{H}$
- 19: **end if**
- 20: **if** $U(\hat{S}_{i+1}, T) > U(S_{\text{fb}}, T)$ **then**
- 21: $S_{\text{fb}} \leftarrow \hat{S}_{i+1}$
- 22: **end if**
- 23: $S_{\text{cur}} \leftarrow \hat{S}_{i+1}$ and $e_{i+1} \leftarrow \hat{e}_{i+1}$
- 24: **end for**
- 25: **return** S_{fb} and the full candidate history $\mathcal{H}$

The revision trace z_i is stored with the candidate and contains the verifier contract, failure ledger, preserve ledger, selected principles, ignored principles, execution anchors, and expected acceptance signals. The reported Revision v3 result is therefore the first successful skill in $\mathcal{H}$ under the maxrev3 budget, or a utility fallback if all candidates fail; it is not necessarily the final generated revision.

C Implementation Settings

This appendix records the fixed runtime and retrieval settings needed to reproduce the reported SKILLREVISE runs. Benchmark task bundles and

selection records are described in Appendix A; prompt interfaces are listed in Appendix K.

Execution and selection. Unless otherwise stated, skill authoring and revision use GPT-5.5, while the task executor is the model named in the corresponding result row. Each task starts from an initially authored skill v_0 . The standard setting allows at most $B = 3$ revision rounds and returns the first verifier-passing evaluated skill, with utility fallback if no evaluated skill succeeds. Budget sweeps vary only B .

Retrieval and binding. Principle Memory is fixed before evaluation for SkillsBench, SkillLearnBench-Random, and SWE-Skills-Bench-Hard; online absorption from test tasks is disabled. At each revision step, Diagnosis supplies task metadata, acceptance criteria, labels, causal judgment, rewrite targets, and evidence snippets to a hybrid retriever. The revision prompt receives a small candidate set and must select only evidence-supported principles.

Runtime logging. The LLM wrapper uses deterministic low-temperature generation and retry logic; task-level verifier timeouts are inherited from task metadata. Each evaluated candidate records status, reward, return code, timeout flag, token count, tool calls, steps, latency, selected principles, ignored principles, and execution anchors.

Fixed settings. The seed memory contains seven repair principles. Retrieval combines sparse and dense rankings by reciprocal-rank fusion with $w_s = 0.5$, $w_d = 0.5$, and $\kappa = 60$. Dense scoring uses qwen/qwen3-embedding-4b with metadata-dominant retrieval and content weight 0.05. Up to four principles are injected into the revision prompt, and at most three may be selected. The wrapper uses a 600s HTTP timeout, three retries with 2s base delay, temperature 0.2, and 4096 maximum generation tokens. ALFWorld is the only setting with an absorbed bank, built from a separate 25-task calibration split.

D Cross-Model Transfer Details

Table 7 gives the success counts behind Figure 3 for the four target executors with verified transfer runs. All results use the same 57-task GPT-5.5 source-success subset. *Own Rev. v3* reports the executor-specific Revision v3 result on this subset.

GPT-5.5 skill installs the fixed GPT-5.5-produced final skill artifact for the executor.

Executor	No skill	Own Rev. v3	GPT-5.5 skill
Opus-4.7	16/57 (28.07)	41/57 (71.93)	33/57 (57.89)
DeepSeek-V4-Pro	14/57 (24.56)	38/57 (66.67)	27/57 (47.37)
Kimi-2.6	16/57 (28.07)	33/57 (57.89)	19/57 (33.33)
Qwen-3.6-Plus	6/57 (10.53)	26/57 (45.61)	11/57 (19.30)

Table 7: Detailed data for Figure 3. Values are successes over the 57-task GPT-5.5 source-success subset with success rates in parentheses.

Analysis. The fixed GPT-5.5-produced skills improve over no-skill execution for all four target executors, but the size of the transfer gain varies substantially. The largest gains appear for Opus-4.7 (+17 successes) and DeepSeek-V4-Pro (+13 successes), while Kimi-2.6 and Qwen-3.6-Plus show smaller gains of +3 and +5 successes. This indicates that revised skills contain reusable procedural guidance, but their utility depends on how well the target executor follows the source skill’s tool-use and verification conventions. Executor-specific revision remains stronger in every row: *Own Rev. v3* exceeds the transferred GPT-5.5 skill by 8 successes for Opus-4.7, 11 for DeepSeek-V4-Pro, 14 for Kimi-2.6, and 15 for Qwen-3.6-Plus. Thus, direct transfer is useful as a portability signal, but adapting skills to the target executor remains important.

E Domain-Level SkillsBench Breakdown

Table 8 reports the SkillsBench domain-level success counts for GPT-5.5 and Opus-4.7, separating no-skill execution, one-shot Skill Creator, and Revision v3.

Domain	GPT-5.5			Opus-4.7		
	No	Creator	Rev	No	Creator	Rev
Energy	1/4	1/4	3/4	1/4	1/4	3/4
Office	6/14	4/14	9/14	2/14	3/14	8/14
Math	5/10	6/10	8/10	4/10	5/10	7/10
Robotics	2/6	5/6	6/6	1/6	5/6	3/6
Software	3/11	5/11	8/11	1/11	4/11	5/11
Science	5/13	4/13	7/13	5/13	6/13	9/13
Media	5/11	3/11	4/11	0/11	1/11	4/11
Finance	2/7	3/7	4/7	1/7	1/7	2/7
Cyber.	2/6	3/6	3/6	1/6	2/6	3/6
Health.	0/1	0/1	0/1	0/1	0/1	0/1
Manuf.	0/3	0/3	1/3	0/3	0/3	0/3

Table 8: Domain-level SkillsBench success counts for GPT-5.5 and Opus-4.7. Each cell reports successful tasks over evaluated tasks. Creator denotes the one-shot Skill Creator baseline, Rev denotes Revision v3, and Cyber., Health., and Manuf. abbreviate Cybersecurity, Healthcare, and Manufacturing.

Analysis. The domain breakdown shows that the aggregate SkillsBench gains are not concentrated in a single task family. For GPT-5.5, Revision v3 improves over no-skill execution in 9 of 11 domains, accounting for the overall gain from 31 to 53 successes. The largest absolute increases occur in Software (3/11 to 8/11) and Robotics (2/6 to 6/6), followed by Office and Math (+3 successes each). This suggests that revision is especially useful when failures come from missing procedural steps, fragile tool use, or incomplete verifier-facing checks. The main exception is Media, where GPT-5.5 drops from 5/11 without skills to 4/11 after revision, indicating that skill conditioning can still hurt when direct exploration is already competitive.

The Opus-4.7 breakdown shows a similar but not identical pattern. Revision v3 improves over no-skill execution in 9 of 11 domains and yields the largest gains in Office (2/14 to 8/14), Science (5/13 to 9/13), Media (0/11 to 4/11), and Software (1/11 to 5/11). These improvements explain why Opus-4.7 has the largest aggregate SkillsBench gain in Table 1. However, the Robotics row also shows a useful failure case: Skill-Creator reaches 5/6 while Revision v3 reaches 3/6, so iterative revision is not uniformly dominant at the domain level. This reinforces the need for explicit selection and for analyzing domain-level regressions rather than relying only on aggregate success.

F Case Studies

We analyze representative GPT-5.5 SkillsBench tasks because this setting has matched no-skill, Skill Creator, and maxrev3 SKILLREVISE runs for all evaluated tasks. Let B denote the no-skill baseline, C denote Skill Creator, and R denote SKILLREVISE maxrev3. Table 9 lists the three outcome patterns requested for analysis; importantly, R is the selected skill under the revision budget and does not include no-skill fallback.

Pattern	Count	Representative task	Selected
$B = 0, C = 0, R = 1$	13/86	react-performance-debugging	v_1
$B = 1, C = 0, R = 1$	7/86	exceltable-in-ppt	v_1
$B = 1, C = 0, R = 0$	2/86	video-filler-word-remover	v_0

Table 9: Representative GPT-5.5 SkillsBench case-study patterns. B is no skill, C is Skill Creator, and R is SKILLREVISE maxrev3.

Run	Evidence and revision signal
No skill	Fail. The agent does not satisfy the checkout-latency verifier, and no reusable non-regression checklist is available to prioritize the failing performance contract.
Skill Creator	Fail. The one-shot skill gives broad Next.js optimization advice but remains too generic, with brittle literals and no bounded fallback path.
SKILLREVISE v_0	Fail. Checkout takes 916ms, above the required 800ms threshold, while UI, bundle, rerender, cart, compare-page, and test-id checks pass. Diagnosis labels false certainty, over-specificity, context pollution, and wrong abstraction level while preserving the passed checks.
SKILLREVISE v_1	Pass; selected. The verifier succeeds with outcome score 1.0. The revised skill adds endpoint-specific measurement, fallback timing checks, preserve constraints for test-observed UI behavior, and a smallest-failing-check rerun before broader verification.

Table 10: Version-level trace for the web-performance case study.

Worked success example: a revised skill completes the task.

In react-performance-debugging, the initial Revision v3 skill v_0 fails the checkout-latency verifier: the checkout path takes 916ms, above the required 800ms threshold, even though the UI, bundle, rerender, cart, compare-page, and test-id checks pass. Table 10 shows the corresponding version-level trace. Diagnosis attributes the failure to a skill that is too literal and too certain, with no fallback path when the first optimization plan is insufficient. The selected revised skill v_1 succeeds with outcome score 1.0 and overall utility 1.042.

Successful revised skill excerpt

Selected version: v_1 , *Next.js Web Performance Non-Regression Workflow*.

Core revised checkpoints:

- Discover project contracts before editing: scripts, tests, routes, components, timing thresholds, and invariants that already pass.
- Localize the bottleneck with measured evidence from tests, request timing, build output, logs, or profiling; fall back to the closest available check when a tool is missing.
- Apply the smallest targeted fix, such as removing artificial waits, repeated expensive work, sequential independent awaits, oversized payloads, or per-request recomputation.
- Verify in increasing scope: rerun the smallest latency check first, then broader build and verifier tests, while preserving data-testid attributes and instrumentation.

Revision recovers failures when a task needs a grounded workflow.

In react-performance-debugging, both no-skill execution and Skill Creator fail, while SKILLREVISE succeeds with v_1 . The failed initial traces expose a typical skill-design problem: the guidance is too generic for a Next.js performance task

that must simultaneously improve latency, preserve behavior, and pass UI tests. Diagnosis flags missing fallback handling, over-specific literals, context pollution, and false certainty. The selected revision reframes the skill as a non-regression workflow: discover local scripts and tests, localize the bottleneck with measurable evidence, preserve test IDs and user-facing behavior, and rerun the relevant verifier before finalizing. This case illustrates the core benefit of revision: verifier feedback turns a broad optimization skill into an execution checklist that constrains the agent toward targeted, behavior-preserving edits.

Revision can repair skill-induced errors even when no skill already succeeds.

In `exceltable-in-ppt`, no-skill execution succeeds, Skill Creator fails, and SKILLREVISION succeeds with v_1 . The Skill Creator and initial v_0 failures are not due to lack of task ability; the verifier reports that an unrelated spreadsheet cell is corrupted, including a non-target CNY-to-EUR value changing from 0.127 to NaN. The revision therefore adds operational safeguards: snapshot embedded workbook cells before editing, identify the exact row and column from slide evidence, update only the target non-formula cell, compare the reopened output against the pre-write snapshot, and retry with a lower-impact workbook-part replacement if serialization corrupts unrelated cells. This pattern shows that SKILLREVISION is not merely giving the executor more instructions. It can make a useful skill less hazardous by converting a high-level edit recipe into a preservation-aware procedure.

Some no-skill successes are still harmed by skill guidance.

In `video-filler-word-remover`, no-skill execution succeeds, but both Skill Creator and SKILLREVISION fail. The verifier indicates that the produced filler-word clip duration falls outside the accepted range, so the failure is a calibration error in segment detection rather than a missing output file. Across the revision episode, the method repeatedly diagnoses context pollution and false certainty and attempts to add validation and fallback rules, but no revised candidate succeeds and the fallback remains v_0 . This is a residual limitation of skill-conditioned execution: when a task can be solved by direct instance-specific exploration, a generic media-processing workflow can over-regularize the agent into a brittle ASR/timestamping pipeline. Because `maxrev3` does not fall back to no-skill execu-

tion, this case remains a failure for SKILLREVISION.

G ALFWorld Additional Results

Table 11 reports the task-family success rates for the ALFWorld run in Section 5.4. The totals match the selected 71/100 run reported in the main text.

ALFWorld task family	Success	SR (%)
Look at object in light	18/21	85.7
Pick and place	13/19	68.4
Clean then place	6/15	40.0
Cool then place	13/18	72.2
Heat then place	14/16	87.5
Pick two and place	7/11	63.6
Overall	71/100	71.0

Table 11: ALFWorld task-family success rates for SKILLREVISION with Qwen3-8B on the 100-task evaluation set.

H Failure Modes

Against Skill Creator, the standard SKILLREVISION `maxrev3` setting wins by success on 20 tasks, ties on 65, and loses on 1; by overall utility, it wins on 58 tasks, ties on 2, and loses on 26. The gap between success and overall utility reflects an important failure mode: some revised skills solve the task but increase execution cost. Other failures arise when Diagnosis attributes environment or model-capability failures to the skill, when a revision overfits to the observed verifier, or when the required fix is a specialized domain method absent from the current Principle Memory.

H.1 Quantified failure taxonomy

We summarize the GPT-5.5 SkillsBench `maxrev3` revision traces to characterize recurring failure types. Across 86 tasks, the trace set contains 208 candidate-level diagnosis entries. The coarse diagnosis labels are dominated by skill-form defects: `context_pollution` appears in 207 entries, `false_certainty` in 181, and both `over_specificity` and `wrong_abstraction_level` in 54 entries; `environment_mismatch` appears once. Table 12 reports the more operational failure types stored in the structured failure ledger for revision attempts. Primary counts are single-label, while secondary mentions are multi-label and therefore do not sum to the number of records.

Leakage class	Rejected from retained artifacts	Allowed during current episode	Allowed retained guidance	Audit check
Task answer leakage	Final answers, exact patches, reference-skill text, target output values, or copied solution steps.	Task instructions, visible files, execution traces, and verifier feedback for the active repair.	Output-format checks, validation routines, and general execution order.	Reusable without the audited instance.
Instance identifiers	Hard-coded local paths, filenames, IDs, constants, or values not required as reusable behavior.	Task-declared paths, schemas, and assets when they are discovered or supplied by the current task.	Instructions to discover task-declared paths, schemas, and assets at runtime.	No frozen one-off literals.
Verifier leakage	Hidden tests, reward-script shortcuts, expected outputs, magic thresholds, or guessed oracle behavior.	Observable verifier contracts and failure messages exposed in the current trace.	Verifier-facing checks stated as format, tolerance, sentinel, or rerun procedures.	Does not infer hidden oracle facts.
Overfit repair	A procedure whose trigger or action only matches the audited instance.	Local diagnosis evidence that explains why the current skill failed.	A condition-action rule that transfers to sibling tasks with different files or values.	Trigger names the failure type, not the instance.
Principle leakage	Absorbed memory that copies task facts, answers, or benchmark-specific literals.	Calibration or revision evidence before filtering and abstraction.	A compact repair principle with an evidence-backed trigger, operation, and transfer guard.	Principle remains task-family level.

Table 13: Manual leakage audit checklist for retained skills, revision traces, and absorbed principles. The wide form separates current-episode evidence use from retained reusable guidance.

H.2 Skill-induced regression

Using Boolean task success, one-shot Skill Creator regresses 9 of the 31 no-skill successes.

Failure type and typical signal	Prim.	Sec.	Tasks
Method / algorithmic workflow <i>Wrong solver, media, numeric, or artifact-generation procedure.</i>	46	23	34
Verification gap <i>Output partly passes tests but misses a verifier-critical check.</i>	18	66	41
Schema / artifact format <i>Invalid JSON/CSV/XLSX structure, fields, ranges, or serialization.</i>	11	22	20
Constraint / efficiency cost <i>Preserved behavior, word budget, token cost, or latency is violated.</i>	11	5	9
Data understanding <i>Semantic answer, counting, extraction, or interpretation is wrong.</i>	8	8	9
Overfit / hard-code <i>Repair depends on over-specific literals or instance-shaped shortcuts.</i>	2	10	9
Incomplete execution <i>Required file, cell range, or processing stage is not produced.</i>	1	0	1
Environment / tool / path <i>Tool, environment, or output-path alignment is implicated.</i>	0	3	3
Unknown / untyped <i>Revision metadata lacks a structured failure ledger.</i>	27	0	17

Table 12: Failure taxonomy for GPT-5.5 SkillsBench maxrev3 revision attempts. Primary counts cover 124 revision records with metadata; secondary counts are multi-label mentions; Tasks counts unique tasks with either a primary or secondary mention.

SKILLREVISE maxrev3 reduces this to 2 no-skill successes that become failures, and it regresses 1 Skill-Creator success. These remaining regressions are the cases where a skill-conditioned execution is still worse than direct task exploration, motivating fallback-aware selection or routing in future work.

I Manual Leakage Audit

We audit evaluation-time skill leakage using the boundary summarized in Table 13: whether a retained skill or absorbed principle functions as reusable guidance rather than as a task solution (Li et al., 2026). This is distinct from pretraining contamination. SKILLREVISE is allowed to use task instructions, execution traces, and verifier feedback during the current revision episode, but the retained artifact should not encode task-instance answers, hidden verifier facts, or one-off shortcuts that would solve only the audited instance.

I.1 Audit scope

We manually inspected sampled retained artifacts across SkillsBench, SkillLearnBench-Random, SWE-Skills-Bench-Hard, and ALFWorld. The inspected units include selected revised skills, revision traces used to justify edits, and absorbed ALFWorld principles. Raw task logs and transient failed candidates may contain instance-local evidence because they are diagnosis inputs; the audit target is the content retained for later use, transfer, or principle retrieval.

I.2 Benchmark-specific boundaries

For SkillsBench and SkillLearnBench-Random, task instructions and visible environment files can be used as current-task context, but retained skills must phrase paths, schemas, and values as discovery or validation steps rather than as memorized constants. For SWE-Skills-Bench-Hard, repository prompts often name files, tests, or issue-specific behavior; using such information during the current repair is expected, while retaining a repository-specific patch recipe or reference-skill content as a general skill is treated as leakage. For ALFWorld,

absorbed principles may describe interaction-level repairs such as object tracking or admissible-action recovery, but should not memorize a room layout, object placement, or trajectory from a calibration task.

I.3 Outcome and residual risk

The audited retained artifacts did not reveal direct answer leakage. When revision traces contained task-specific observations, we kept them as provenance for the current candidate but required the retained skill or principle to rewrite them into reusable discovery, validation, fallback, or preservation rules. This manual audit cannot prove that every transient raw candidate was leakage-free, and verifier-visible feedback can still encourage overfitting to observable checks. We therefore treat the audit as a qualitative safeguard alongside the main protocol controls: no online absorption from test tasks for the three verifier benchmarks, verifier re-execution before selection, and conservative filtering before ALFWorld principle absorption.

J Additional Limitations

Verifier dependence. SKILLREVISE improves skills from execution traces and verifier-facing failures, so its revisions are only as informative as the available feedback. More generally, verifier-driven skill evolution relies on visible pass/fail signals that may be only partial proxies for the hidden requirements that determine success. Sparse tests, opaque scoring scripts, or proxy checks can therefore lead Diagnosis to repair visible assertions rather than the underlying behavior. We rely on benchmark-provided verifiers and task traces rather than an auxiliary learned judge, which avoids adding another judge but limits feedback density.

Revision cost and stopping. The method trades additional model calls, tool use, and wall-clock time for better skills. Although maxrev3 and success-prioritized early stopping limit this cost, some successful revisions still solve tasks through longer or more expensive procedures. A fixed budget is also imperfect: easy tasks may need no revision, while hard tasks may require a domain method that cannot be recovered within three attempts. Adaptive stopping, cheaper diagnosis, and earlier detection of misleading skills remain important directions.

Scope and generalization. Our evaluation covers multiple executors and three verifier-based

benchmarks, but it remains centered on tasks with reproducible tests or artifact validators. It also leaves realistic long-horizon agent benchmarks such as AgentVista and Tool Decathlon outside our scope (Su et al., 2026; Li et al., 2025). The results do not establish behavior in long-running deployments, safety-critical settings, adversarial tasks, or environments where the agent defines its own success criteria. In addition, maxrev3 does not fall back to no-skill execution, so skill conditioning can still hurt tasks that are better solved by direct instance-specific exploration. Future work should study larger skill libraries, dynamic routing, and deployment settings with changing tools, distributions, and user preferences.

K Prompt Templates

This appendix reports the prompt and prompt-like interfaces used by the implementation. We use the same method-level authoring and revision templates across SkillsBench, SkillLearnBench-Random, and SWE-Skills-Bench-Hard; only the task-specific placeholder contents differ by benchmark and instance. Task-specific instructions, execution traces, and retrieved principles are represented as placeholders. Platform API configuration, credentials, and provider-specific hidden system prompts are omitted because they are not part of the proposed revision operator.

Skill authoring prompts.

Initial Skill Authoring Prompt

Instruction. You are writing a task-family skill for an LLM agent. Use platform-agnostic skill-authoring constraints: write a reusable procedure, keep only behavior-changing guidance, make workflow order and checkpoints explicit, validate inputs/files/tools/assumptions before acting, ground actions in the current environment, include fallback handling, add strict constraints, and avoid hard-coded paths, versions, or commands unless they are verified invariants.

Inputs. [TASK_FAMILY], [SOURCE_TASK], [ACCEPTANCE_CRITERIA], and optionally [RETRIEVED_PRINCIPLES].

Principle use. Use relevant principles as operating guidance, not as text to copy. The final skill must not mention principle IDs or copy task-specific answers, hidden verifier values, one-off constants, or unverified paths.

Output. Return only Markdown with sections # <Skill Name>, Purpose, When to Use, Procedure, and Constraints / Pitfalls.

Task execution prompts.

SkillsBench/Codex Task Execution Prompt

Instruction. You are executing one SkillsBench task inside the provided workspace. Complete the task by editing or creating the required files in this workspace. Do not ask follow-up questions. Use local files and validation commands when available. When finished, provide a concise summary of what changed and what validation you ran.

Optional skill guidance. If [INSTALLED_SKILL] is nonempty, use it as execution guidance. Treat it as reusable guidance, not as a task answer.

Inputs. [TASK_INSTRUCTION] and optionally [INSTALLED_SKILL].

Baseline use. The no-skill baseline uses this execution prompt with an empty [INSTALLED_SKILL]. Skill Creator and SKILLREVISE use the same execution prompt after installing the generated or revised skill.

Benchmark-Native Execution Interface

Used for. SkillLearnBench-Random and SWE-Skills-Bench-Hard run through their benchmark-native executor interfaces after export into the shared task-directory harness.

Interface. The executor receives a task directory containing `instruction.md`, environment files, tests or verifier scripts, and task metadata. When a skill is present, the harness materializes it as the benchmark skill source or skill directory; otherwise the same task runs with no installed skill.

Prompt provenance. The benchmark or agent framework supplies its normal task prompt from the task interface. We do not add a second method-specific task-solving prompt beyond installing the skill artifact and passing the same task instruction, environment, and verifier contract.

ALFWorld Interactive Action Prompt

Static context. You are solving an ALFWorld text-only task. At each turn choose one valid environment action, preferably exactly one of the admissible commands. Explore only as needed, track object and receptacle names exactly, and stop after the environment reports success. The context includes [TASK_INSTRUCTION], [PROBLEM_PATH], and optionally reusable skill guidance.

Per-turn inputs. [TURN_INDEX], [MAX_TURNS], optional compact trajectory memory, [CURRENT_OBSERVATION], and [ADMISSIBLE_COMMANDS].

Action policy. Choose exactly one admissible command; do not repeat a failed action unless alternatives are exhausted. If actions alternate between the same locations, choose a new admissible destination or inspect a new relevant object. For look-at-object-in-light tasks, after taking the target object, actively search unvisited destinations for a lamp or DeskLamp.

Output. Return exactly one action string and no prose.

Diagnosis prompts.

LLM Diagnosis Prompt

Instruction. Diagnose why an LLM-authored skill helps, hurts, or fails to help a task execution. Use paired execution evidence, not stylistic preference alone.

Inputs. [TASK_ID], [TASK_FAMILY], [TASK_INSTRUCTION], [ACCEPTANCE_CRITERIA], [CURRENT_SKILL], authoring-prior violations, [NO_SKILL_TRACE], [WITH_SKILL_TRACE], and [UTILITY].

Allowed labels. path, schema, method, environment, incomplete execution, data understanding, constraint violation, verification gap, efficiency timeout, overfit or hardcoded, context pollution, false certainty, wrong abstraction level, and related skill-defect labels used by the harness.

Output. Return only JSON with keys labels, evidence, causal_judgment, rewrite_targets, and summary. Evidence items contain source, snippet, and reason. Non-LLM diagnosis runs construct the same fields deterministically from traces and verifier outcomes.

Skill revision prompts.

Structured Revision Prompt

Instruction. Revise this task-family skill using execution evidence and Principle Memory. The revised skill should teach a reusable non-regression workflow, not memorize this task's answer. Use selected principles as repair operators, not as extra prose pasted into the skill.

Inputs. [PREVIOUS_REVISION_TRACE], [TASK_FAMILY], [TASK_INSTRUCTION], [ACCEPTANCE_CRITERIA], [CURRENT_SKILL], [DIAGNOSIS_LABELS], [CAUSAL_JUDGMENT], [EVIDENCE], [REWRITE_TARGETS], and top-*k* [RETRIEVED_PRINCIPLES].

Protocol. Build a verifier contract, failure ledger, and preserve ledger before editing. Select at most three retrieved principles, and for each selected principle state why it applies, which failed check/evidence it addresses, what skill operation it induces, and what preserve constraint it protects. If the same failure repeats across attempts, escalate from local patching to method-level repair. Every revision must introduce or preserve an execution anchor with action, expected evidence, and placement. Edit only the responsible trigger, procedure, or constraint unless evidence implicates shared setup. Remove task-instance answers, brittle literals, hidden-verifier guesses, and instructions that only work for the observed instance.

Output. Return exactly two blocks: first REVISION_TRACE_JSON, then REVISED_SKILL_MARKDOWN.

REVISION_TRACE_JSON Schema

The structured revision prompt requires a machine-readable trace before the revised skill. The required fields are:

- **verifier_contract:** observable requirements only, including required output paths, schemas or formats, numeric thresholds, and pass/fail assertions. Unknown requirements must remain unknown rather than guessed.
- **failure_ledger:** failed checks, actual behavior, likely cause, and a lightweight failure type such as path, schema, method, environment, incomplete execution, data understanding, constraint violation, verification gap, efficiency timeout, overfit or hardcoded, or unknown.
- **preserve_ledger:** passed checks and successful choices to keep. Preserved behavior should not be removed, renamed, contradicted, or replaced unless it directly conflicts with the failed check.
- **selected_principles:** for each selected principle, principle ID, reason for selection, addressed failure, induced skill operation, and preserve constraint.
- **ignored_principles:** retrieved principles not used, with evidence-grounded reasons.
- **repeated_failure_escalation:** whether repeated failure triggered a higher-level repair and what escalation action was taken.
- **execution_anchors:** concrete action, expected observable evidence, and placement in the revised skill.
- **acceptance_signals:** expected utility improvement, expected failed assertions reduced, and preserve risk.

The second block must be the revised skill Markdown with the same standard sections used by the initial authoring prompt.

Ablation prompts.

Free-form Revision Prompt

Instruction. Improve this LLM-authored task skill using the observed execution feedback. Do not use any predefined failure taxonomy, repair-principle checklist, or structured defect labels. Keep useful parts of the current skill, remove misleading parts, and make the revised skill reusable. Do not regress verifier checks that already passed while addressing the observed feedback.

Inputs. [TASK_FAMILY], [TASK_INSTRUCTION], [ACCEPTANCE_CRITERIA], [CURRENT_SKILL], and [OBSERVED_EXECUTION_FEEDBACK].

Output. Return only the revised skill Markdown with sections # <Skill Name>, Purpose, When to Use, Procedure, and Constraints / Pitfalls.

LLM Backend Wrapper Prompt

Instruction. You are the LLM backend for SKILLREVISE; follow the requested output format exactly and add no unrelated commentary.

Purpose field. The wrapper receives a purpose such as skill_authoring, skill_revision, skill_revision_freeform, or skill_diagnosis, then passes the corresponding method prompt as [PROMPT].

Command-line LLM variant. Return only the final answer; do not include progress notes or run tools unless necessary.

L Principle Memory Entries

The seed Principle Memory contains reusable repair operators rather than task answers. Each entry has a trigger, repair operation, and guard against over-specific retention.

Workflow checkpointing

Trigger: broad advice without verifiable intermediate decisions.

Repair: rewrite the procedure as ordered checkpoints: discover, validate, act, verify, and recover.

Guard: encode the check, not a one-task answer.

Input schema validation

Trigger: malformed fields, wrong types, missing keys, or numeric mismatches.

Repair: reload produced artifacts and assert required keys, types, shapes, ranges, and serialization formats.

Guard: check task-declared schema, not expected values.

Environment-output grounding

Trigger: expected artifact is missing or written outside the checked path.

Repair: check task-specified output paths and inspect mounts, permissions, scripts, or supported write routes if needed.

Guard: use exact paths only when supplied or locally discovered.

Verifier-contract alignment

Trigger: plausible domain logic misses a verifier convention.

Repair: restate sentinels, formats, tolerances, traversal rules, or scoring rules as pre-finalization checks.
Guard: record the convention class, not hidden answers.

Fallback after tool failure

Trigger: a command, tool, file, endpoint, or assumption fails without a valid alternative.
Repair: add one bounded fallback branch, choose the closest supported alternative, and rerun the smallest check.
Guard: keep fallback short and task-directed.

Transfer-preserving repair

Trigger: repair targets one instance, path, literal, or answer rather than reusable behavior.
Repair: replace instance-specific content with a trigger condition and reusable decision rule.
Guard: require usefulness for sibling tasks with different files or values.

Trigger noninterference

Trigger: skill guidance adds cost or steers execution when it is not needed.
Repair: narrow *When to Use* and add exclusions so the skill fires only when it changes an execution decision.
Guard: prefer concise trigger boundaries over broad domain labels.

L.1 Principle-Guided Repair Contrasts

We analyze five principle-guided contrasts involving the seed principles in Appendix L. In each contrast, the standard run selects a revised skill containing the principle and succeeds, while the corresponding w/o Principles ablation fails. The boxes below pair the weak or failed skill content with the corresponding principle-guided revision. We omit task answers, raw verifier values, hidden expected outputs, and instance-specific assertions.

Input schema validation.

Weak skill: schema-shaped output

Extract the handbook rules, write the requested JSON artifacts, and run the provided checks. Treat a parseable file with the expected top-level fields as ready unless a command explicitly fails.

Revised skill: reload and recompute

Write artifacts atomically; reload each produced JSON file; check required keys, types, ordering, and rounding; then independently recompute the task-critical metric from the task-visible inputs before finalizing.

Analysis. In the manufacturing maintenance contrast, the standard run selects v_1 and succeeds, while w/o Principles selects v_0 and later no-principle candidates continue to fail on a task-critical numeric artifact. The weak skill validates that the artifact looks like the requested output, but it does not prove that the computation inside the artifact is correct. The revised skill changes the failure boundary: a syntactically valid but semantically wrong file becomes a blocked pre-finalization state, forcing the executor to repair the computation method instead of merely producing well-formed JSON.

Fallback after tool failure.

Weak skill: continue after missing evidence

Search for the required itinerary entities, record the tools used, and continue with the best available plan when one source or route lookup is unavailable.

Revised skill: bounded fallback

Maintain a required-evidence ledger. If a route or source is missing, inspect available tools, choose a verified equivalent, rerun only the affected search, and preserve already-passing itinerary constraints.

Analysis. In the travel planning contrast, the standard run selects v_3 and succeeds, while w/o Principles selects v_1 and fails despite containing a generic checklist for searched itinerary construction. The weak skill treats missing evidence as something to work around with a plausible plan. The revised skill instead treats it as a local recovery problem. This works be-

cause the repair is neither a full replanning pass nor a guess: it is a bounded fallback that keeps the valid parts of the itinerary stable while recovering the specific missing evidence.

Transfer-preserving repair.

Weak skill: instance-shaped patch

After observing a failure, patch the immediate case with concrete local filenames, literals, or one-task assumptions that explain the observed mismatch.

Revised skill: reusable repair rule

Discover paths and schemas from the current task; preserve handbook-grounded formulas, sorting, rounding, and schema checks; avoid storing instance identifiers, expected values, or one-task constants.

Analysis. The same manufacturing maintenance contrast shows why transfer preservation matters. The standard v_1 succeeds, whereas w/o Principles stays at v_0 as the selected skill and later no-principle revisions keep failing on the same method-level defect. The weak repair could solve the observed failure by memorizing the instance. The principle-guided repair instead expresses the fix as a task-family procedure: discover the local contract, apply the reusable computation rule, and validate the emitted artifact. That is why the revised skill remains a skill rather than an answer patch.

Workflow checkpointing.

Weak skill: solve then report

Parse the network, solve the dispatch problem, write the report, and run a final check after the report has been produced.

Revised skill: checkpointed workflow

Run discover, solve, pre-write validate, write, reload, and post-write verify as separate checkpoints. Recompute report totals, loading, reserve, margin, and cost from the emitted artifact.

Analysis. For the grid dispatch contrast, the standard run selects v_2 and succeeds, while w/o Principles selects v_0 and its later candidate still fails after producing a report that is not economically consistent. The weak skill contains the right domain ingredients, but it does not connect the solver state to the serialized artifact. Checkpointing makes the final report itself the object that must satisfy the contract, so objective drift or serialization drift is caught even when earlier feasibility checks look plausible.

Trigger noninterference.

Weak skill: broad domain trigger

Use grid-dispatch guidance whenever the task appears related to power systems, and include general domain reasoning before producing the final report.

Revised skill: narrow execution trigger

Trigger only for structured DC-dispatch report tasks. Keep contract discovery, optimization, serialized-report checks, and bounded recovery; exclude unrelated grid analysis or narrative-only tasks.

Analysis. Grid dispatch also illustrates how trigger scope affects repair. The no-principle selected skill is broad enough to guide a dispatch attempt, but it does not focus the executor on the final report gate. The principle-guided revision removes low-signal domain prose and narrows the trigger to the exact class of tasks where the skill changes an execution decision. This reduces skill-induced distraction and lets the executor spend its budget on the checks that determine success.

Across these contrasts, principles convert verifier feedback into reusable repair operations rather than task answers, helping the standard run recover failures that the w/o Principles ablation continues to rewrite around.

M GPT-5.5 Per-Task Heatmap



Figure 4: Per-task verifier outcome heatmap for GPT-5.5 across methods on SkillsBench. Rows are grouped by revision-response pattern, placing tasks solved by all methods before tasks newly solved by revision, partially improved tasks, and persistently difficult tasks. Columns compare the no-skill baseline, Skill Creator, and revised skills under revision budgets Rev1–Rev5. Darker sage-blue cells indicate higher verifier outcome scores, near-white cells indicate low-scoring or failed executions, and thin horizontal separators mark task groups.

N Opus-4.7 SkillLearnBench Per-Task Heatmap

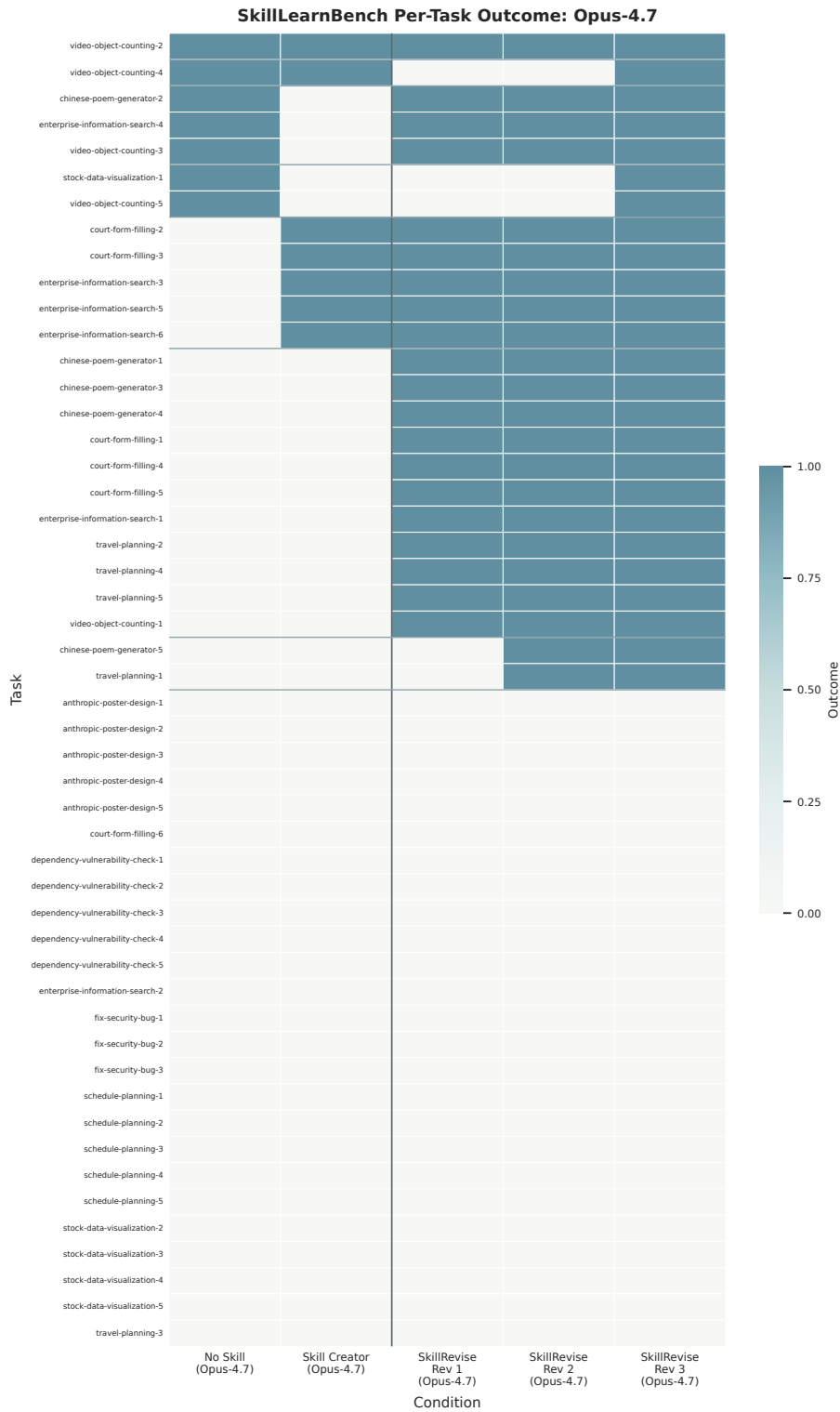


Figure 5: Per-task verifier outcome heatmap for Opus-4.7 across methods on SkillLearnBench-Random. Columns compare the no-skill baseline, Skill Creator, and budgeted SKILLREVISE selections after Rev1, Rev2, and Rev3 reconstructed from the recorded revision history. Rows cover the full 50-task selected suite. Rows are sorted to concentrate revision-side successes in the upper-right region while leaving persistent failures at the bottom.

O Qwen-3.6-Plus SWE-Skills-Bench Per-Task Heatmap



Figure 6: Per-task verifier outcome heatmap for Qwen-3.6-Plus across methods on SWE-Skills-Bench-Hard. Columns compare the no-skill baseline, Skill Creator, and budgeted SKILLREVISE selections after Rev1, Rev2, and Rev3 reconstructed from the recorded revision history. Rows are sorted by the binary response pattern so shared successes, revision-only successes, and persistent failures are visually grouped.